



CodeNicely Software Services

Raipur, Chhattisgarh • codenicely.in

FULL-STACK DEVELOPER ASSIGNMENT

Loan Products Admin Panel with Eligibility Engine

Role: Full-Stack Developer (Frontend + Backend)

Suggested Duration: 4 – 6 days

Submission: Git repository + README + short demo video / hosted link



1. Overview

Build an internal admin panel for a lending business. It manages a catalogue of loan products (each with its own eligibility criteria) and a directory of users whose status (Active or Rejected) is set automatically by running their details against every product's rules.

When an admin adds a user, the system evaluates them against each product. If the user qualifies for at least one product they become Active; otherwise Rejected. Each active user can be inspected to see which products they qualify for.

2. Technical Expectations

You may use any stack you are comfortable with. The following are suggestions, not hard requirements — choose what lets you ship clean, well-structured code.

Layer	Suggested Options
Frontend	React / Next.js / Vue / Angular (any modern SPA framework)
Backend	Node.js (Express/NestJS) / Django / FastAPI / Spring Boot / Laravel
Database	PostgreSQL / MySQL / MongoDB
Auth	JWT or session-based auth with role-based access control
API	REST or GraphQL, documented

Non-negotiables:

- Clear separation between frontend, backend, and database layers.
- All eligibility logic must live on the backend (never trust the client).
- Permissions enforced on the server, not just hidden in the UI.
- Eligibility evaluation must be deterministic and re-runnable.

3. Roles & Permissions

Implement at least two roles. Permissions must be enforced both in the UI (show/hide or disable controls) and on the backend (reject unauthorized requests).

Role	Products Tab	Users Tab
Admin	Full access: view, add, edit products & eligibility criteria	Full access: view, add users, view status & eligible products
Viewer / Manager	View products only (no add/edit)	View users & statuses only (cannot add)



The key requirement: add/edit on products and add on users must be permission-gated.

4. Functional Requirements

4.1 Tab 1 — Products

A management screen listing all loan products. Users with the right permission can add and edit products along with their eligibility criteria.

- List all loan products in a table/grid with key attributes.
- Add Product — create a new loan product and define its eligibility criteria.
- Edit Product — modify an existing product's details and criteria.
- Editing criteria must **re-evaluate all users** so statuses stay correct (see section 6).

Each loan product should include at least the following eligibility criteria:

Criterion	Type	Example Rule
Age (from DOB)	Numeric range	Min 21, Max 58 years
Credit Score	Numeric min	Minimum 700
Employment Type	Enum	Salaried OR Self-Employed (or both)
Salary Type	Enum	DAT, Cash, or Cheque (allowed set)
Salary	Numeric min	Minimum ₹25,000 / month

Notes on the model:

- Age is derived from Date of Birth at evaluation time, not stored as a static number.
- Employment Type and allowed Salary Types should be multi-select per product (a product may accept more than one).
- A product is satisfied only when the user meets ALL of its criteria.

4.2 Tab 2 — Users

A directory of users. Admins can add a user and their details; the system then runs the eligibility engine and files the user under the correct status.

- Add User — capture the details below; on save, run eligibility against every product.
- **Active Users** — user qualifies for at least one product.
- **Rejected Users** — user qualifies for no product.
- Show the two groups clearly (separate lists, tabs, or a status column/filter).
- Each **active user** has a button (e.g. “View Eligible Products”) that opens the list of products they qualify for.

User details to capture:



Field	Type	Used For
Full Name	Text	Display
Date of Birth	Date	Age calculation
Credit Score	Number (300–900)	Score check
Employment Type	Salaried / Self-Employed	Employment match
Salary Type	DAT / Cash / Cheque	Salary-type match
Salary (monthly)	Number	Salary check

5. Eligibility Engine (Core Logic)

This is the heart of the assignment. Implement it on the backend as a reusable, testable function.

For a given user, evaluate against each product:

1. Compute the user's age from DOB.
2. For each product, check every criterion:
 - age within [minAge, maxAge]
 - creditScore $\geq$ product.minCreditScore
 - employmentType is in product.allowedEmploymentTypes
 - salaryType is in product.allowedSalaryTypes
 - salary $\geq$ product.minSalary
3. The user is eligible for the product only if ALL criteria pass.
4. Collect every product the user passes.
5. If the eligible list is non-empty $\rightarrow$ status = **Active**; otherwise $\rightarrow$ status = **Rejected**.

Worked example:

	Product A (Personal)	Product B (Business)
Age 21–58	✓ (user 30)	✓ (user 30)
Credit $\geq$ 700	✓ (user 720)	✓ (user 720)
Employment	Salaried ✓	Self-Employed only ✗
Salary Type	Cheque ✓	Cheque ✓
Salary $\geq$ min	✓	✓
Result	ELIGIBLE	NOT ELIGIBLE

Outcome: user qualifies for Product A $\rightarrow$ Active. “View Eligible Products” shows Product A.

6. Re-Evaluation Requirement

Statuses must never go stale. Re-run eligibility whenever the underlying data changes:

- When a new user is added $\rightarrow$ evaluate that user.
- When a product's criteria are edited $\rightarrow$ re-evaluate all users.
- When a new product is added $\rightarrow$ re-evaluate all users (someone rejected before may now qualify).



A user previously Rejected may become Active after a product is loosened or added, and vice versa. Handle these transitions correctly.

7. Deliverables

1. Source code in a Git repository (frontend + backend).
2. README with setup steps, how to run, tech choices, and login credentials.
3. Database schema/migrations plus seed data (a few products and users covering active and rejected cases).
4. Optional: a short demo video or a hosted URL of the working app.

Tip: prioritise a correct, well-tested eligibility engine and clean re-evaluation over UI polish. The logic is what we care about most.